

stats101a_hw9

Jaden Chou - 206654728

2026-05-28

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.0      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.2      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(car)
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
## The following object is masked from 'package:dplyr':
##
##     recode
##
## The following object is masked from 'package:purrr':
##
##     some
```

```
library(leaps)
```

(1)

```
df <- data.frame(read.csv('./pgatour2006-3.csv'))
str(df)
```

```
## 'data.frame':   196 obs. of  12 variables:
```

```
## $ Name : chr "Aaron Baddeley" "Adam Scott" "Alex Aragon" "Alex Cejka" ...
## $ TigerWoods : int 0 0 0 0 0 0 0 0 0 0 ...
## $ PrizeMoney : int 60661 262045 3635 17516 16683 107294 50620 57273 86782 23396 ...
## $ AveDrivingDistance: num 288 301 303 289 288 ...
## $ DrivingAccuracy : num 60.7 62 51.1 66.4 63.2 ...
## $ GIR : num 58.3 69.1 59.1 67.7 64 ...
## $ PuttingAverage : num 1.75 1.77 1.79 1.78 1.76 ...
## $ BirdieConversion : num 31.4 30.4 29.9 29.3 29.3 ...
## $ SandSaves : num 54.8 53.6 37.9 45.1 52.4 ...
## $ Scrambling : num 59.4 57.9 50.8 54.8 57.1 ...
## $ BounceBack : num 19.3 19.4 16.8 17.1 18.2 ...
## $ PuttsPerRound : num 28 29.3 29.2 29.5 28.9 ...
```

```
df <- df |>
  select(-TigerWoods, -Name)
```

```
str(df)
```

```
## 'data.frame': 196 obs. of 10 variables:
## $ PrizeMoney : int 60661 262045 3635 17516 16683 107294 50620 57273 86782 23396 ...
## $ AveDrivingDistance: num 288 301 303 289 288 ...
## $ DrivingAccuracy : num 60.7 62 51.1 66.4 63.2 ...
## $ GIR : num 58.3 69.1 59.1 67.7 64 ...
## $ PuttingAverage : num 1.75 1.77 1.79 1.78 1.76 ...
## $ BirdieConversion : num 31.4 30.4 29.9 29.3 29.3 ...
## $ SandSaves : num 54.8 53.6 37.9 45.1 52.4 ...
## $ Scrambling : num 59.4 57.9 50.8 54.8 57.1 ...
## $ BounceBack : num 19.3 19.4 16.8 17.1 18.2 ...
## $ PuttsPerRound : num 28 29.3 29.2 29.5 28.9 ...
```

(a)

```
best_fit <- regsubsets(
  PrizeMoney ~ .,
  data = df,
  nvmax = ncol(df) - 1
)

best_summary <- summary(best_fit)

best_model_index <- which.max(best_summary$adjr2)
best_summary$which[best_model_index, ]
```

```
##      (Intercept) AveDrivingDistance DrivingAccuracy GIR
##      TRUE      FALSE      TRUE      TRUE
##      PuttingAverage BirdieConversion SandSaves Scrambling
##      FALSE      TRUE      TRUE      TRUE
##      BounceBack PuttsPerRound
##      FALSE      FALSE
```

```

final_model <- lm(
  PrizeMoney ~ DrivingAccuracy +
    GIR +
    BirdieConversion +
    SandSaves +
    Scrambling,
  data = df
)

summary(final_model)

##
## Call:
## lm(formula = PrizeMoney ~ DrivingAccuracy + GIR + BirdieConversion +
##     SandSaves + Scrambling, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -80972 -26436  -6308   17398  420690
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -1103461.1    109276.9  -10.098 < 2e-16 ***
## DrivingAccuracy    -1848.5       815.7   -2.266  0.02457 *
## GIR              10135.8       1480.8    6.845  1.03e-10 ***
## BirdieConversion  10274.1       1715.0    5.991  1.03e-08 ***
## SandSaves         1172.7        736.4    1.592  0.11297
## Scrambling       4445.7        1454.5    3.057  0.00256 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 49880 on 190 degrees of freedom
## Multiple R-squared:  0.4063, Adjusted R-squared:  0.3907
## F-statistic: 26.01 on 5 and 190 DF,  p-value: < 2.2e-16

```

$\text{PrizeMoney} = \beta_0 + \beta_1(\text{DrivingAccuracy}) + \beta_2(\text{GIR}) + \beta_3(\text{BirdieConversion}) + \beta_4(\text{SandSaves}) + \beta_5(\text{Scrambling}) + \epsilon$

(b)

```

best_fit <- regsubsets(
  PrizeMoney ~ .,
  data = df,
  nvmax = ncol(df) - 1
)

best_summary <- summary(best_fit)

best_bic_index <- which.min(best_summary$bic)
best_summary$which[best_bic_index, ]

```

##	(Intercept)	AveDrivingDistance	DrivingAccuracy	GIR
##	TRUE	FALSE	TRUE	TRUE
##	PuttingAverage	BirdieConversion	SandSaves	Scrambling
##	FALSE	TRUE	FALSE	TRUE
##	BounceBack	PuttsPerRound		
##	FALSE	FALSE		

$$\text{PrizeMoney} = \beta_0 + \beta_1(\text{DrivingAccuracy}) + \beta_2(\text{GIR}) + \beta_3(\text{BirdieConversion}) + \beta_4(\text{Scrambling}) + \varepsilon$$

This one does not include the Scrambling feature.

(c)

```
forward_fit <- regsubsets(
  PrizeMoney ~ .,
  data = df,
  method = "forward",
  nvmax = ncol(df) - 1
)
```

```
forward_summary <- summary(forward_fit)
```

```
best_adjr2 <- which.max(forward_summary$adjr2)
```

```
forward_summary$which[best_adjr2, ]
```

##	(Intercept)	AveDrivingDistance	DrivingAccuracy	GIR
##	TRUE	FALSE	TRUE	TRUE
##	PuttingAverage	BirdieConversion	SandSaves	Scrambling
##	FALSE	TRUE	TRUE	TRUE
##	BounceBack	PuttsPerRound		
##	FALSE	FALSE		

Best model using adjusted R-squared:

$$\text{PrizeMoney} = \beta_0 + \beta_1(\text{DrivingAccuracy}) + \beta_2(\text{GIR}) + \beta_3(\text{BirdieConversion}) + \beta_4(\text{SandSaves}) + \beta_5(\text{Scrambling}) + \varepsilon$$

```
best_bic <- which.min(forward_summary$bic)
```

```
forward_summary$which[best_bic, ]
```

##	(Intercept)	AveDrivingDistance	DrivingAccuracy	GIR
##	TRUE	FALSE	TRUE	TRUE
##	PuttingAverage	BirdieConversion	SandSaves	Scrambling
##	FALSE	TRUE	FALSE	TRUE
##	BounceBack	PuttsPerRound		
##	FALSE	FALSE		

Best model using BIC:

$$\text{PrizeMoney} = \beta_0 + \beta_1(\text{DrivingAccuracy}) + \beta_2(\text{GIR}) + \beta_3(\text{BirdieConversion}) + \beta_4(\text{Scrambling}) + \varepsilon$$

(d)

```
backward_fit <- regsubsets(
  PrizeMoney ~ .,
  data = df,
  method = "backward",
  nvmax = ncol(df) - 1
)

backward_summary <- summary(backward_fit)

best_adjr2 <- which.max(backward_summary$adjr2)

backward_summary$which[best_adjr2, ]
```

##	(Intercept)	AveDrivingDistance	DrivingAccuracy	GIR
##	TRUE	FALSE	TRUE	TRUE
##	PuttingAverage	BirdieConversion	SandSaves	Scrambling
##	FALSE	TRUE	TRUE	TRUE
##	BounceBack	PuttsPerRound		
##	FALSE	FALSE		

Best model using adjusted R-squared:

$$\text{PrizeMoney} = \beta_0 + \beta_1(\text{DrivingAccuracy}) + \beta_2(\text{GIR}) + \beta_3(\text{BirdieConversion}) + \beta_4(\text{SandSaves}) + \beta_5(\text{Scrambling}) + \varepsilon$$

```
best_bic <- which.min(backward_summary$bic)

backward_summary$which[best_bic, ]
```

##	(Intercept)	AveDrivingDistance	DrivingAccuracy	GIR
##	TRUE	FALSE	TRUE	TRUE
##	PuttingAverage	BirdieConversion	SandSaves	Scrambling
##	FALSE	TRUE	FALSE	TRUE
##	BounceBack	PuttsPerRound		
##	FALSE	FALSE		

Best model using BIC:

$$\text{PrizeMoney} = \beta_0 + \beta_1(\text{DrivingAccuracy}) + \beta_2(\text{GIR}) + \beta_3(\text{BirdieConversion}) + \beta_4(\text{Scrambling}) + \varepsilon$$

(e)

variables that repeatedly appear: - DrivingAccuracy - GIR - BirdieConversion - Scrambling

some: - SandSaves

(2)

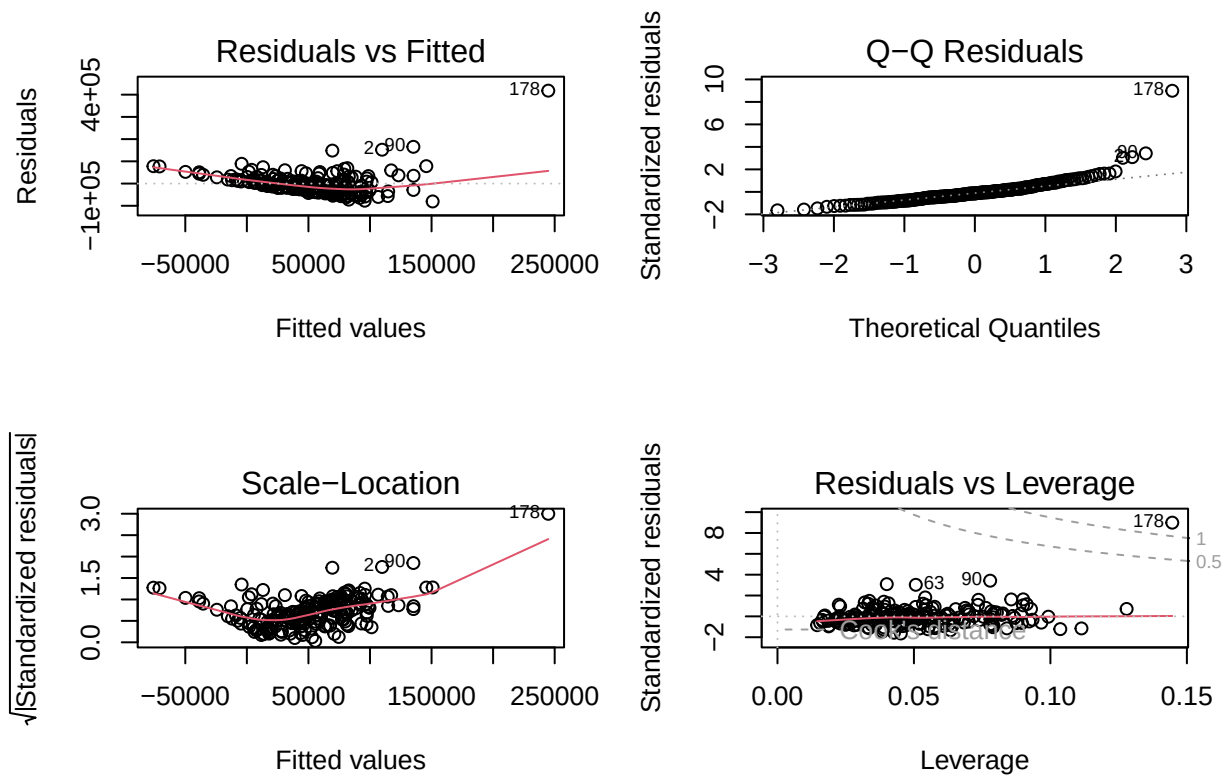
(a)

```
full_model <- lm(PrizeMoney ~ ., data = df)
summary(full_model)

##
## Call:
## lm(formula = PrizeMoney ~ ., data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -80475 -26186  -6671  15209 417966
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -1174845.3   591253.0  -1.987  0.04839 *
## AveDrivingDistance    -720.8     766.0  -0.941  0.34795
## DrivingAccuracy    -2457.9    1104.0  -2.226  0.02720 *
## GIR             10709.3     3761.8   2.847  0.00491 **
## PuttingAverage   123072.1    579670.7   0.212  0.83209
## BirdieConversion   11758.4     3801.9   3.093  0.00229 **
## SandSaves         1074.7      759.4   1.415  0.15868
## Scrambling         4313.5     2477.8   1.741  0.08336 .
## BounceBack          568.5     1585.2   0.359  0.72028
## PuttsPerRound       701.0     37306.6   0.019  0.98503
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 50270 on 186 degrees of freedom
## Multiple R-squared:  0.4097, Adjusted R-squared:  0.3811
## F-statistic: 14.34 on 9 and 186 DF,  p-value: < 2.2e-16

par(mfrow = c(2, 2))

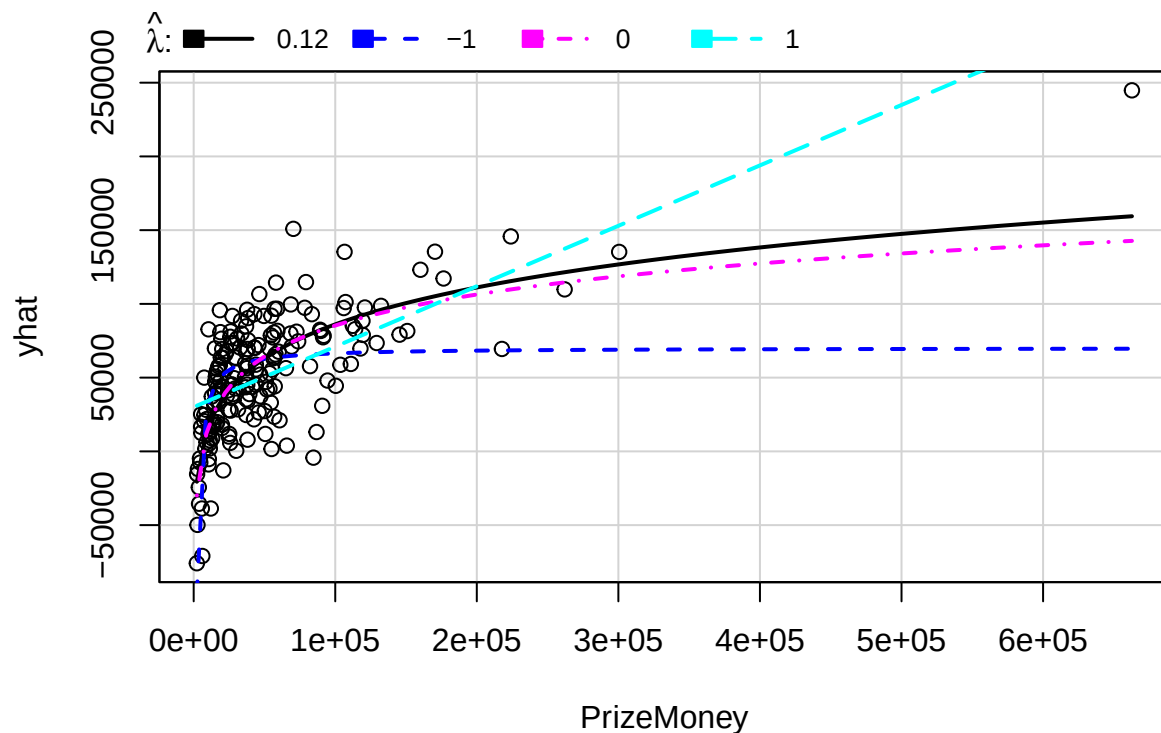
plot(full_model)
```



There's a trend in residuals v. fitted that shows breaks the linear trend. constant variance is broken (fanning in residuals v. fitted and increasing trend in scale-location) QQ plot is not the best, errors are not standard.

(b)

```
invResPlot(full_model)
```



```
##      lambda      RSS
## 1  0.1151362 152996786636
## 2 -1.0000000 202164262328
## 3  0.0000000 153655411582
## 4  1.0000000 192579012441
```

log transform is quite close to the optimal lambda.

```
summary(powerTransform(full_model))
```

```
## bcPower Transformation to Normality
##      Est Power Rounded Pwr Wald Lwr Bnd Wald Up Bnd
## Y1    0.0332          0    -0.0706          0.137
##
## Likelihood ratio test that transformation parameter is equal to 0
## (log transformation)
##              LRT df      pval
## LR test, lambda = (0) 0.3923114  1 0.53109
##
## Likelihood ratio test that no transformation is needed
##              LRT df      pval
## LR test, lambda = (1) 336.417  1 < 2.22e-16
```

the recommended rounded power is 0. With the LR test, lambda = 0 p value > 0.05, so we do not reject. The LR test for lambda 1 has pvalue of < 2.22e-16 < 0.05, so we reject the null hypothesis , concluding we do need a transformation.

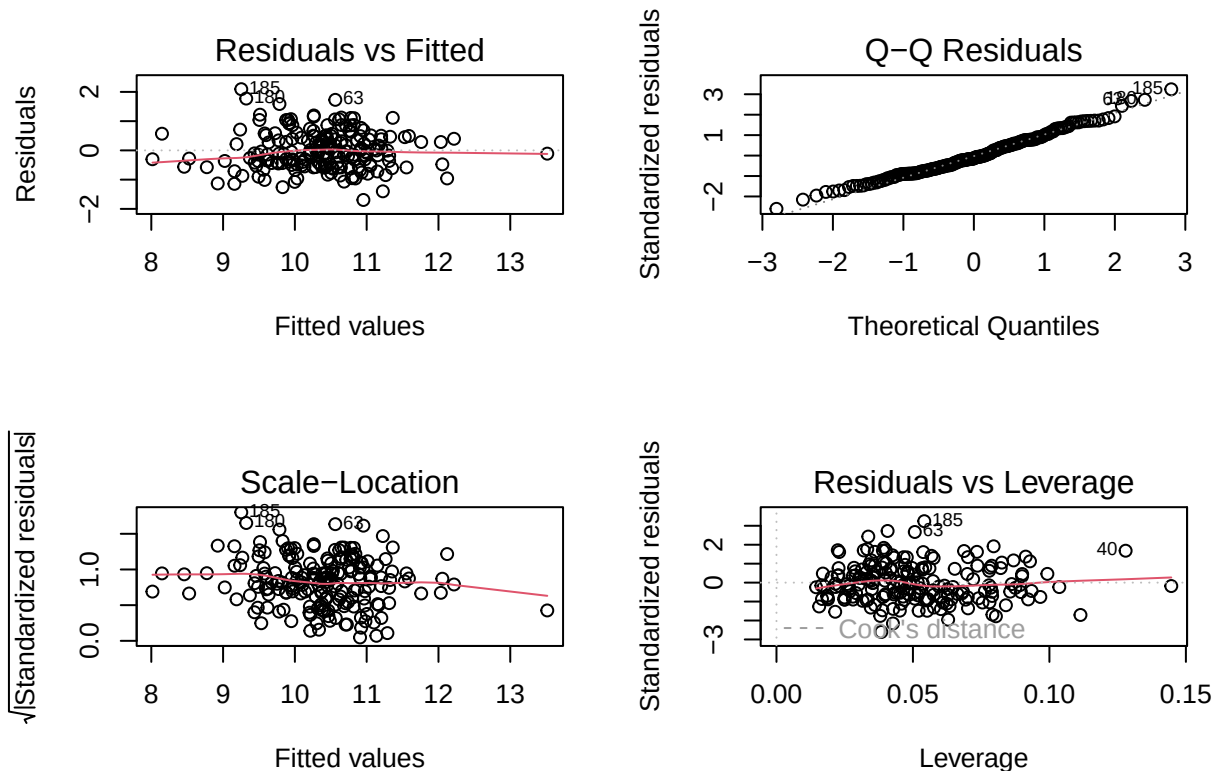
log transform is good.

(c)

```
log_model <- lm(log(PrizeMoney) ~ ., data = df)

par(mfrow = c(2, 2))

plot(log_model)
```



Validity is improved. There is a much lesser indication of a trend in Residuals v. Fitted. And constant variance seems to be good (or at the very least much better) in residuals vs. fitted and scale-location plot. Q-Q residuals looks much more straight line as well.

(d)

```
pt_pred <- powerTransform(
  cbind(AveDrivingDistance,
        DrivingAccuracy,
        GIR,
        PuttingAverage,
        BirdieConversion,
```

```

        SandSaves,
        Scrambling,
        BounceBack,
        PuttsPerRound) ~ 1,
  data = df
)

summary(pt_pred)

```

```

## bcPower Transformations to Multinormality
##
##      Est Power Rounded Pwr Wald Lwr Bnd Wald Up Bnd
## AveDrivingDistance   -0.8263      1   -3.5754      1.9228
## DrivingAccuracy      -0.1455      0   -1.1997      0.9088
## GIR                   1.3075      1   -0.0450      2.6600
## PuttingAverage       1.1825      1   -3.0744      5.4393
## BirdieConversion     0.9249      1   -0.0112      1.8611
## SandSaves            1.0414      1    0.1034      1.9795
## Scrambling           0.8668      1   -0.5321      2.2657
## BounceBack           0.7305      1   -0.0579      1.5189
## PuttsPerRound       -0.0489      1   -3.2107      3.1128
##
## Likelihood ratio test that transformation parameters are equal to 0
## (all log transformations)
##
##                                LRT df    pval
## LR test, lambda = (0 0 0 0 0 0 0 0 0) 18.19029  9 0.033029
##
## Likelihood ratio test that no transformations are needed
##
##                                LRT df    pval
## LR test, lambda = (1 1 1 1 1 1 1 1 1)  8.462084  9 0.48833

```

```

df_trans <- df

df_trans$AveDrivingDistance <- bcPower(df$AveDrivingDistance, pt_pred$roundlam[1])
df_trans$DrivingAccuracy    <- bcPower(df$DrivingAccuracy,    pt_pred$roundlam[2])
df_trans$GIR                <- bcPower(df$GIR,                pt_pred$roundlam[3])
df_trans$PuttingAverage     <- bcPower(df$PuttingAverage,     pt_pred$roundlam[4])
df_trans$BirdieConversion   <- bcPower(df$BirdieConversion,   pt_pred$roundlam[5])
df_trans$SandSaves          <- bcPower(df$SandSaves,          pt_pred$roundlam[6])
df_trans$Scrambling         <- bcPower(df$Scrambling,         pt_pred$roundlam[7])
df_trans$BounceBack        <- bcPower(df$BounceBack,        pt_pred$roundlam[8])
df_trans$PuttsPerRound     <- bcPower(df$PuttsPerRound,     pt_pred$roundlam[9])

```

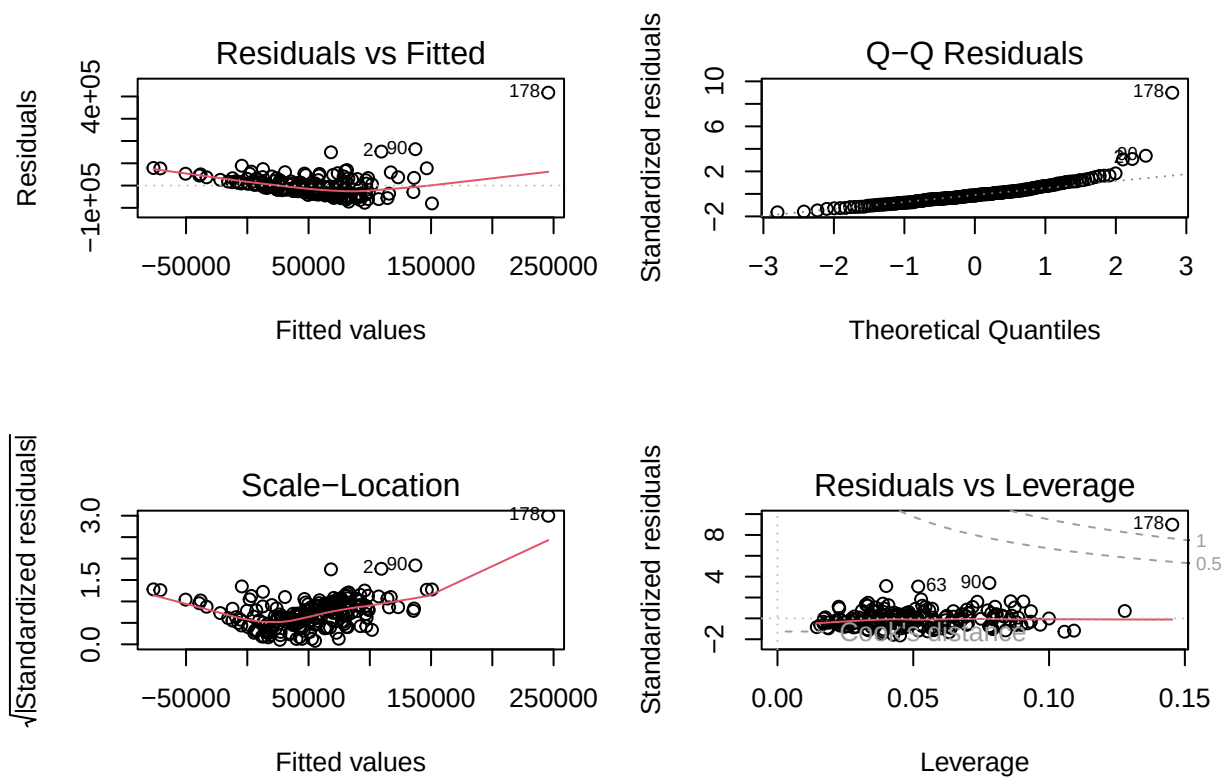
```

bc_model <- lm(
  PrizeMoney ~ .,
  data = df_trans
)

par(mfrow = c(2, 2))

plot(bc_model)

```



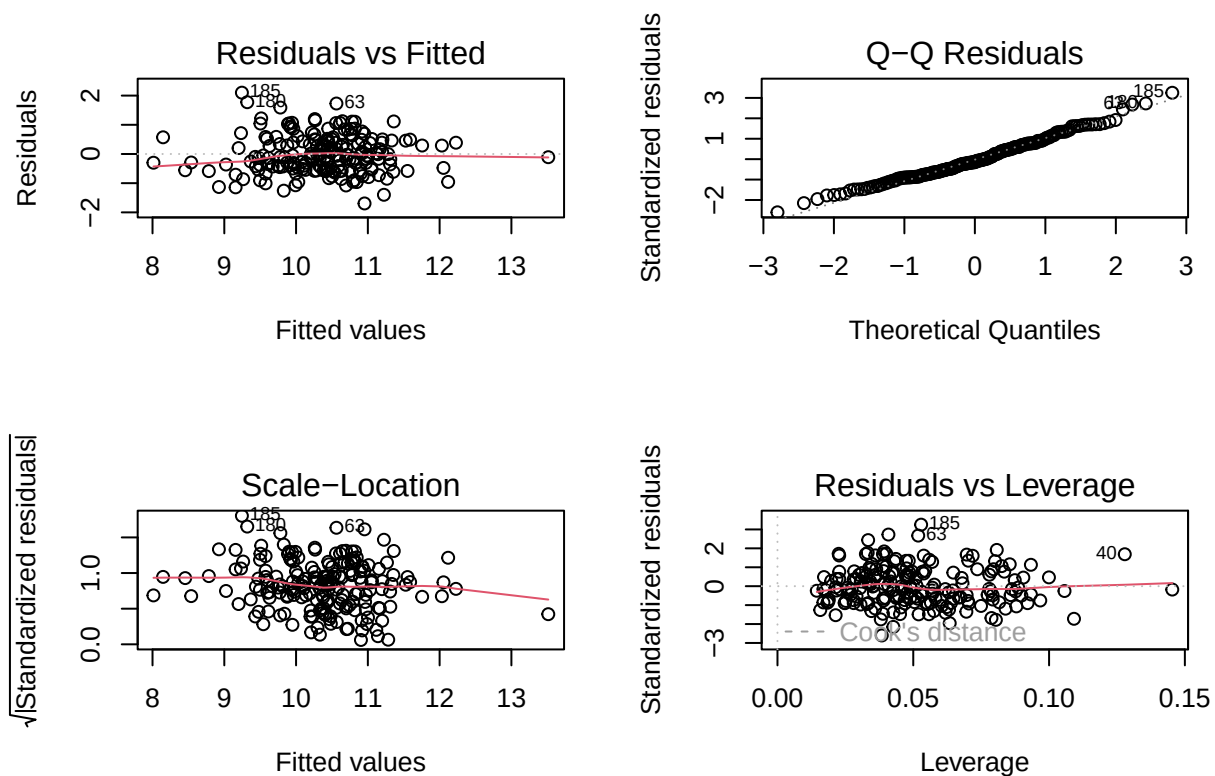
Linearity trend is violated as there exists a trend in the residuals vs fitted plot. Constant variance is violated (fanning in residuals plot and upward trend in scale-location plot). QQ residuals is slightly better but still not amazing.

(e)

```
log_bc_model <- lm(
  log(PrizeMoney) ~ .,
  data = df_trans
)

par(mfrow = c(2, 2))

plot(log_bc_model)
```



I recommend the simply y-transformed model. The y-transformed model and the y and x's transformed model seem very similar, with both of them being much better than the other two models. But, the y-transformed model is easier to interpret compared to the x's being transformed as well, which is why I recommend that one – there's not a large enough improvement in validity for it to be worth the extra complications in interpretation.

(f)

```
hat_values <- hatvalues(log_model)

n <- nrow(df)
p <- length(coef(log_model))

high_lev <- which(hat_values > 2 * p / n)
high_lev
```

```
## 28 40 168 178
## 28 40 168 178
```

28 40 168 178 are observations worth investigating.

```
cooks <- cooks.distance(log_model)

influential <- which(cooks > 4 / n)
influential
```

```
## 1 9 40 47 60 63 84 101 122 125 128 168 180 185
## 1 9 40 47 60 63 84 101 122 125 128 168 180 185
```

1 9 40 47 60 63 84 101 122 125 128 168 180 185 are observations worth investigating.